Data Preprocessing and ETL

Agenda

- What is Data
- Machine Learning
- Data Preprocessing
- Feature Engineering and Extraction
  - Domain knowledge features
  - Date and Time features
  - String operations
  - Web Data
  - Geospatial features
- Feature Transformations
  - Data Cleaning or Cleansing
  - Work with Missing data
  - Work with Categorical data
  - Detect and Handle Outliers
  - Split data to Train and Test Sets
  - Deal with Imbalanced classes
  - Feature Scaling

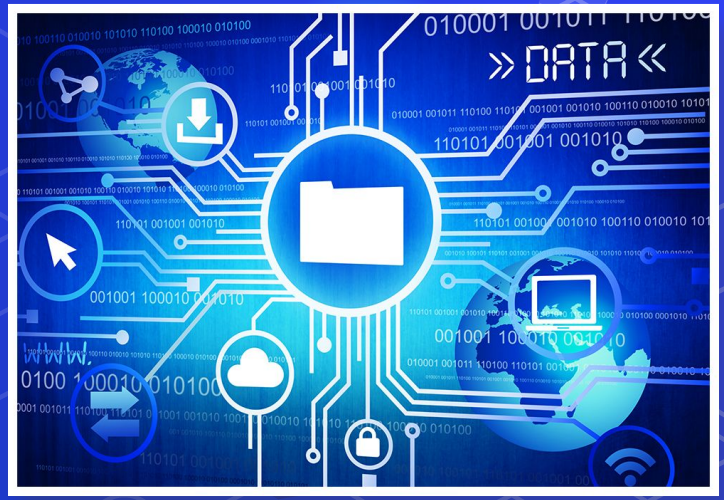
Agenda

- **What is Data**
- Machine Learning
- Data Preprocessing
- Feature Engineering and Extraction
  - Domain knowledge features
  - Date and Time features
  - String operations
  - Web Data
  - Geospatial features
- Feature Transformations
  - Data Cleaning or Cleansing
  - Work with Missing data
  - Work with Categorical data
  - Detect and Handle Outliers
  - Split data to Train and Test Sets
  - Deal with Imbalanced classes
  - Feature Scaling

What is Data

We are living in a data-driven economy. It’s a world where having tons of data, understanding it and knowing what to do with data is power.

Understanding your data is not one of the most difficult things in data science, but it is time-consuming. Interpretation of data is effective when we know about the source of data.



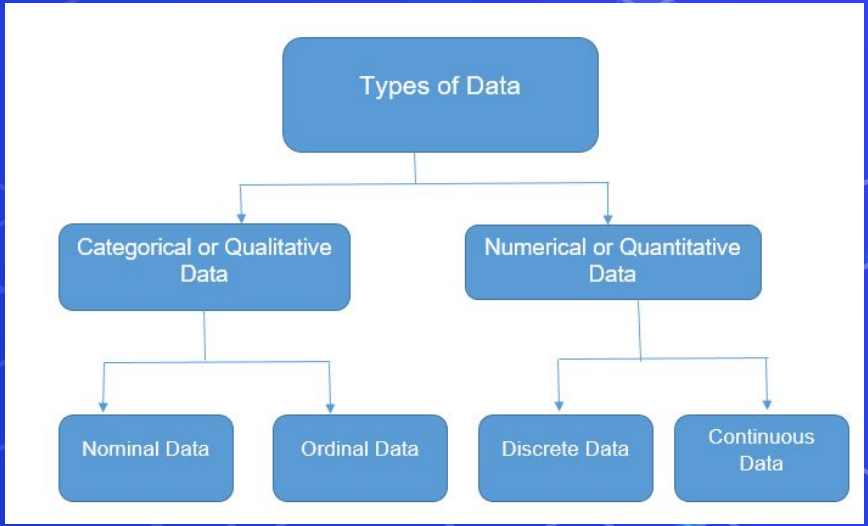
What is Data (Types of Data)

Categorical

This represents qualitative data with no apparent inherent mathematical meaning.
Example: Yes or No, Sex, Race, Marital Status etc.
These can take be assigned numbers like Yes(1) and No(0), but numbers have no mathematical meaning.

Numerical

This represents some sort of quantitative measurement.
Example: height of people, stock price, page load time etc



What is Data (Numerical)

Discrete data
This is integer based, often counts of some event. Example: “How many songs do a user like?” or “How many times a coin flipped to “head” ?”

Continuous data
It has an infinite number of possible values. Example: “How much time did it take for a user to buy?”

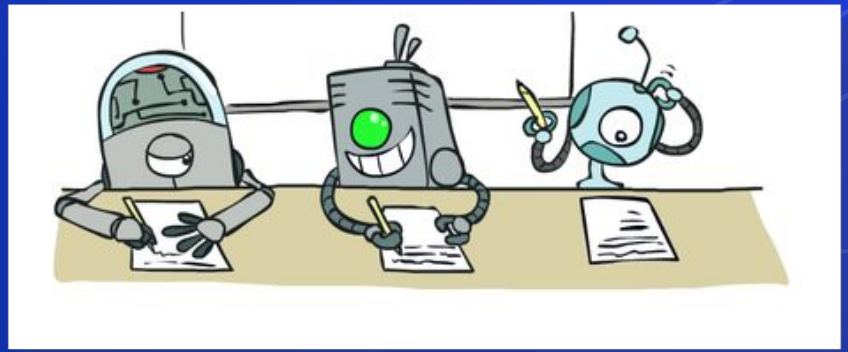
What is Data (Categorical)

Nominal or unordered data
we assign individual items to named categories that do not have an implicit or natural value or rank.
Example: Animals can be cats or dogs or horses etc.. that would be nominal data.

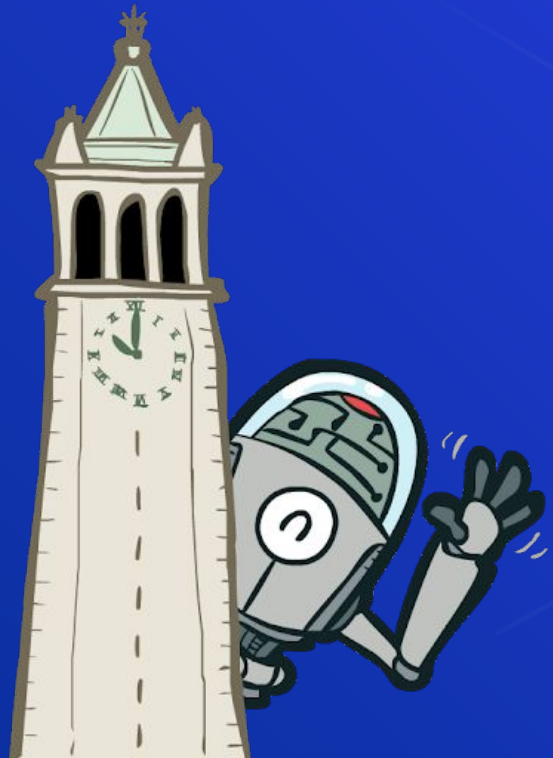
Ordinal or ordered data
items are assigned to categories that do have some kind of implicit or natural order.
Example: “Small, Medium, or Large." Another example is a survey question that asks us to rate an item on a 1 to 10 scale, with 10 being the best. This implies that 10 is better than 9, which is better than 8, and so on.

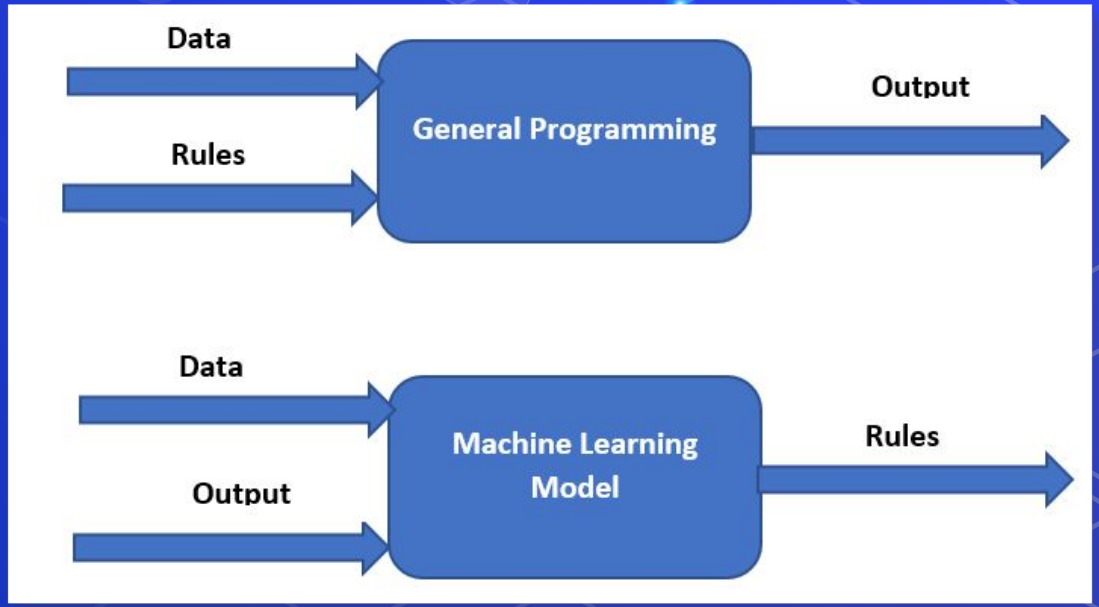
Machine Learning

Machine learning involves computers discovering how they can perform tasks without being explicitly programmed to do so. It involves computers learning from data provided so that they carry out certain tasks.



Machine Learning

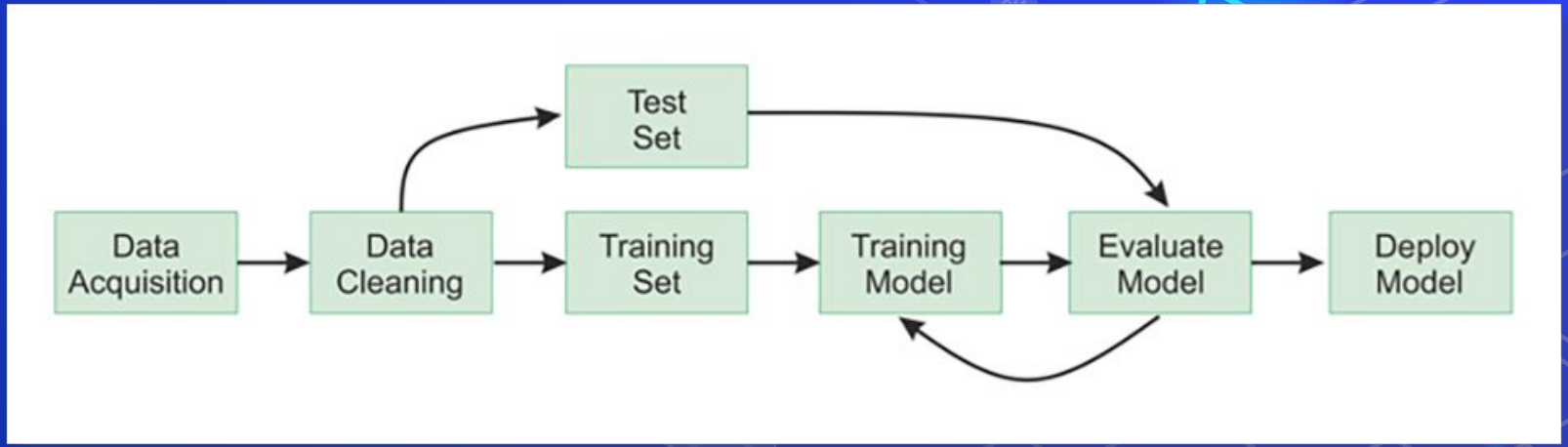




Applications of Machine learning:

1. Image Recognition
2. Speech Recognition.
3. Product recommendations
4. Self-driving cars
5. Email Spam and Malware Filtering

Machine Learning



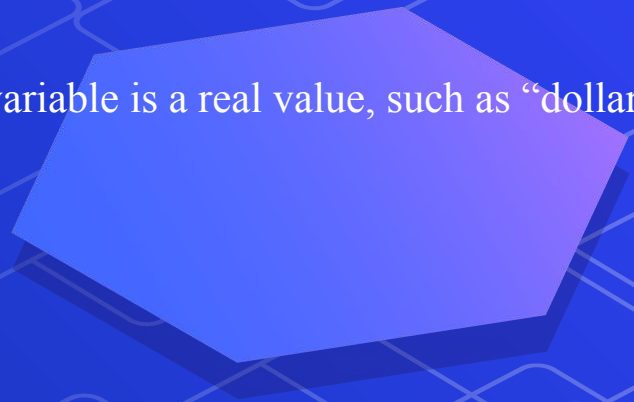
Supervised Learning :

- **For instance**, suppose you are given a basket filled with different kinds of fruits. Now the first step is to train the machine with all different fruits one by one .
- If the shape of the object is rounded and has a depression at the top, is red in color, then it will be labeled as –**Apple**.
- If the shape of the object is a long curving cylinder having Green-Yellow color, then it will be labeled as –**Banana**.
- Now suppose after training the data, you have given a new separate fruit, say Banana from the basket, and asked to identify it.
- Since the machine has already learned the things from previous data. It will first classify the fruit with its shape and color and would confirm the fruit name as BANANA and put it in the Banana category. Thus the machine learns the things from training data(basket containing fruits) and then applies the knowledge to test data(new fruit).

Supervised learning is classified into:

- Classification: A classification problem is when the output variable is a category, such as “Red” or “blue” or “disease” and “no disease”.

- Regression: A regression problem is when the output variable is a real value, such as “dollars” or “weight”



Unsupervised Learning :

- **For instance**, suppose it is given an image having both dogs and cats which it has never seen. .
- Thus the machine has no idea about the features of dogs and cats so we can't categorize it as 'dogs and cats '.
- But it can categorize them according to their similarities, patterns, and differences, i.e., we can easily categorize the above picture into two parts. The first may contain all pics having **dogs** in them and the second part may contain all pics having **cats** in them.

UNSupervised learning is classified into:

- **Clustering**: A clustering problem is where you want to discover the inherent groupings in the data, such as grouping customers by purchasing behavior.

Agenda

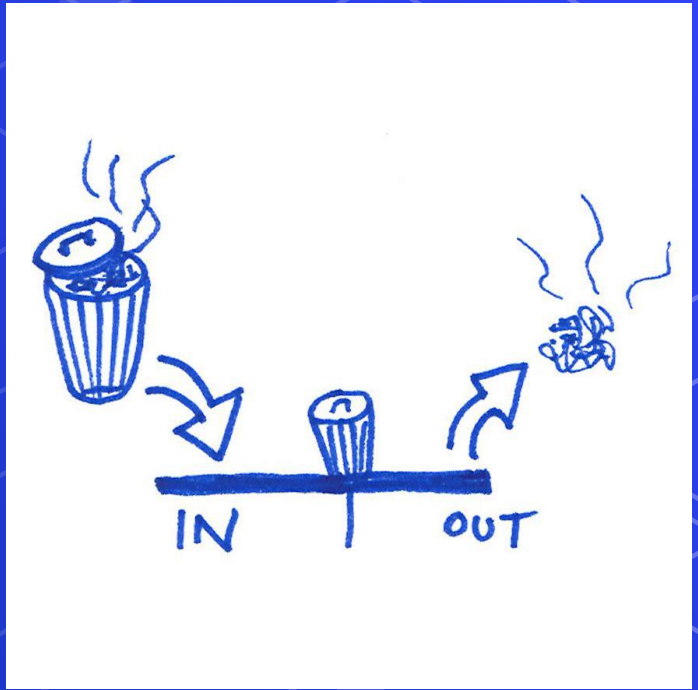
- What is Data
- Machine Learning
- **Data Preprocessing**
- Feature Engineering and Extraction
  - Domain knowledge features
  - Date and Time features
  - String operations
  - Web Data
  - Geospatial features
- Feature Transformations
  - Data Cleaning or Cleansing
  - Work with Missing data
  - Work with Categorical data
  - Detect and Handle Outliers
  - Split data to Train and Test Sets
  - Deal with Imbalanced classes
  - Feature Scaling

Data Preprocessing

The phrase "garbage in, garbage out" is the best description for the use of data preprocessing.

Data gathering methods are often loosely controlled, resulting in out-of-range values, useless features, outliers, missing values, etc.

Garbage data produce misleading results. Thus, data preprocessing is the most important phase of a machine learning project.



ETL

E T L

Extract Transform Load

Data Preprocessing

Feature Engineering and Extraction
Extract and Create useful features from raw data into features suitable for modeling.

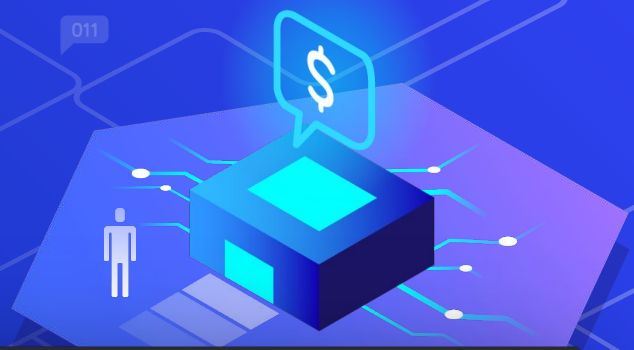
Feature Transformation
Transformation of data to improve the accuracy of the algorithm.

Agenda

- What is Data
- Machine Learning
- Data Preprocessing
- **Feature Engineering and Extraction**
  - **Domain knowledge features**
  - Date and Time features
  - String operations
  - Web Data
  - Geospatial features
- Feature Transformations
  - Data Cleaning or Cleansing
  - Work with Missing data
  - Work with Categorical data
  - Detect and Handle Outliers
  - Split data to Train and Test Sets
  - Deal with Imbalanced classes
  - Feature Scaling

Domain knowledge features

It's the process of creating new useful features from current features based on our understanding of the problem domain knowledge.




```
# example for delivery system we calculate speed in Km/m
df['speed'] = df['Distance (KM)'] / (df['Time from Pickup to Arrival'] / 60)
```

Task 1

Read the tips dataset using seaborn

Find the profit of the restaurant for each bill.

Task 1 Solution

```
import seaborn as sns
df = sns.load_dataset('tips')
df['Profit'] = (df['total_bill'] * 0.86 ) + df['tip']
```

Task 2

Read the sendy_logistics.csv dataset :
Create a new column “Temp_status” that has 3 categories for temperature
1- cold (10 : 21) Celsius degrees
2- good (21 : 30) Celsius degrees
3- hot (larger than 30) Celsius degrees

Task 2 Solution

```
import seaborn as sns
import pandas as pd
df = pd.read_csv('sendy_logistics.csv')
def get_temp(temp):
    if 10 <= temp <=20 :
        return 'cold'
    elif 20 < temp <=30:
        return 'good'
    elif temp > 30 :
        return 'hot'
df['Temp_status'] = df['Temperature'].apply(get_temp)
```

Agenda

- What is Data
- Machine Learning
- Data Preprocessing
- **Feature Engineering and Extraction**
  - Domain knowledge features
  - **Date and Time features**
  - String operations
  - Web Data
  - Geospatial features
- Feature Transformations
  - Data Cleaning or Cleansing
  - Work with Missing data
  - Work with Categorical data
  - Detect and Handle Outliers
  - Split data to Train and Test Sets
  - Deal with Imbalanced classes
  - Feature Scaling

Date and Time features

Date columns usually provide valuable information about the problem, they are neglected as an input for the machine learning algorithms. It might be the reason for this, that dates can be present in numerous formats, which make it hard to understand by algorithms.


```
df['Time'] = pd.to_datetime(df['Time'],format='%d/%m/%Y %H:%M')
```

Date and Time features

```python
df['Year'] = df['Time'].dt.year
df['Month'] = df['Time'].dt.month
df['Month_Name'] = df['Time'].dt.month_name()
df['Week'] = df['Time'].dt.week
df['Day'] = df['Time'].dt.day
df['Week_Day'] = df['Time'].dt.weekday
df['Day_Name'] = df['Time'].dt.day_name()
df['Hour'] = df['Time'].dt.hour
df['Minute'] = df['Time'].dt.minute
```

- **Extract day, hour, minute, seconds, quarter, month, year, etc.**
- Extract time-based features like evenings, noon, night time etc.
- Extract seasonal features like winter, summer, autumn.
- Calculate time elapsed between two related Date features.

Task 1

- Extract from Date months,day name, and the year of each tweet :

Tweets	Date
@alydesigns i was out most of the day so didn't get much done	2004-12-03
@FakerPattyPattz Oh dear. Were you drinking out of the forgotten table drinks?	2018-02-15
@LettyA ahh ive always wanted to see rent love the soundtrack!!	2009-3-01

Task 1 Solution

```
df = pd.read_csv('Task_3_dataset.csv')
df['time'] = pd.to_datetime(df['time'],format='%Y-%m-%d')
df['year'] = df['time'].dt.year
df['month'] = df['time'].dt.month
df['day'] = df['time'].dt.day_name()
```

Date and Time features

```python
def map_hours(x):
    if x in [0,1,2,3,4,5,6,7,8,9,10,11,12]:
        return 'morning'
    elif x in [13,14,15,16]:
        return 'afternoon'
    else:
        return 'evening'

df['Period'] = df['Hour'].apply(map_hours)
```

- Extract day, hour, minute, seconds, quarter, month, year, etc.
- **Extract time-based features like evenings, noon, night time etc.**
- Extract seasonal features like winter, summer, autumn.
- Calculate time elapsed between two related Date features.

Task 2

1. Extract from Time the hour, minutes and seconds of each tweet
2. extract the parts of day[**morning**, **afternoon**, **evening**, **night**] :
   - Morning(0 : 12)
   - Afternoon (13 : 16)
   - Evening (17 : 21)
   - Night (22 : end of the day)

Tweets	Time
@alydesigns i was out most of the day so didn't get much done	2004-12-03 15:08:30
@FakerPattyPattz Oh dear. Were you drinking out of the forgotten table drinks?	2018-02-15 23:17:09
@LettyA ahh ive always wanted to see rent love the soundtrack!!	2009-3-01 03:10:12

Task 2 Solution

```
df = pd.read_csv('Task_4_dataset.csv')
df['time'] = pd.to_datetime(df['time'],format='%Y-%m-%d %H:%M:%S')
df['hour'] = df['time'].dt.hour
df['min'] = df['time'].dt.minute
df['sec'] = df['time'].dt.second
def map_hours(x):
    if x in range(0,13):
        return 'morning'
    elif x in range(13,17):
        return 'afternoon'
    elif x in range (17,22):
        return 'evening'
    else :
        return 'night'
df['Time_status'] = df['hour'].apply(map_hours)
```

Date and Time features

```python
def map_months(x):
    if x in [12, 1, 2]:
        return 'Winter'
    elif x in [3, 4, 5]:
        return 'Spring'
    elif x in [6, 7, 8]:
        return 'Summer'
    elif x in [9, 10, 11]:
        return 'Autumn'

df['Season'] = df['Month'].apply(map_months)
```

- Extract day, hour, minute, seconds, quarter, month, year, etc.
- Extract time-based features like evenings, noon, night time etc.
- **Extract seasonal features like winter, summer, autumn.**
- Calculate time elapsed between two related Date features.

Date and Time features

```
df['Elapsed_Years']  = (datetime.now() - df['Time']) / np.timedelta64(1, 'Y')
```

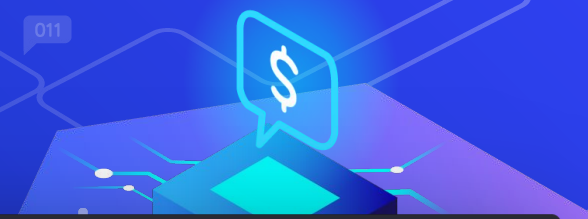
- Extract day, hour, minute, seconds, quarter, month, year, etc.
- Extract time-based features like evenings, noon, night time etc.
- Extract seasonal features like winter, summer, autumn.
- **Calculate time elapsed between two related Date features.**

Agenda

- What is Data
- Machine Learning
- Data Preprocessing
- **Feature Engineering and Extraction**
  - Domain knowledge features
  - Date and Time features
  - **String operations**
  - Web Data
  - Geospatial features
- Feature Transformations
  - Data Cleaning or Cleansing
  - Work with Missing data
  - Work with Categorical data
  - Detect and Handle Outliers
  - Split data to Train and Test Sets
  - Deal with Imbalanced classes
  - Feature Scaling

String operations

There is a lot of data hidden within strings, you can use a lot of string operations techniques to extract those data.



```python
def extract_email_provider(email):
    return email.split('@')[1]

df['Email_Provider'] = df['Email'].apply(extract_email_provider)
```

Multiple Choice

```
txt = "hello, my name is Peter, I am 26 years old"

x = txt.split(", ")

print(x)
```

A. ['hello', 'my name is Peter', 'I am 26 years old’]

A. ['hello',‘my’,‘name’,‘is’,’Peter',‘I’,‘am’,’26’,’years’,’old’]

A. ['hello’]
['my name is Peter’]
['I am 26 years old']

Task 3

- Read the dataset “Sales_Dec_2019.csv”
- Find the most profit Town

Task 3 Solution

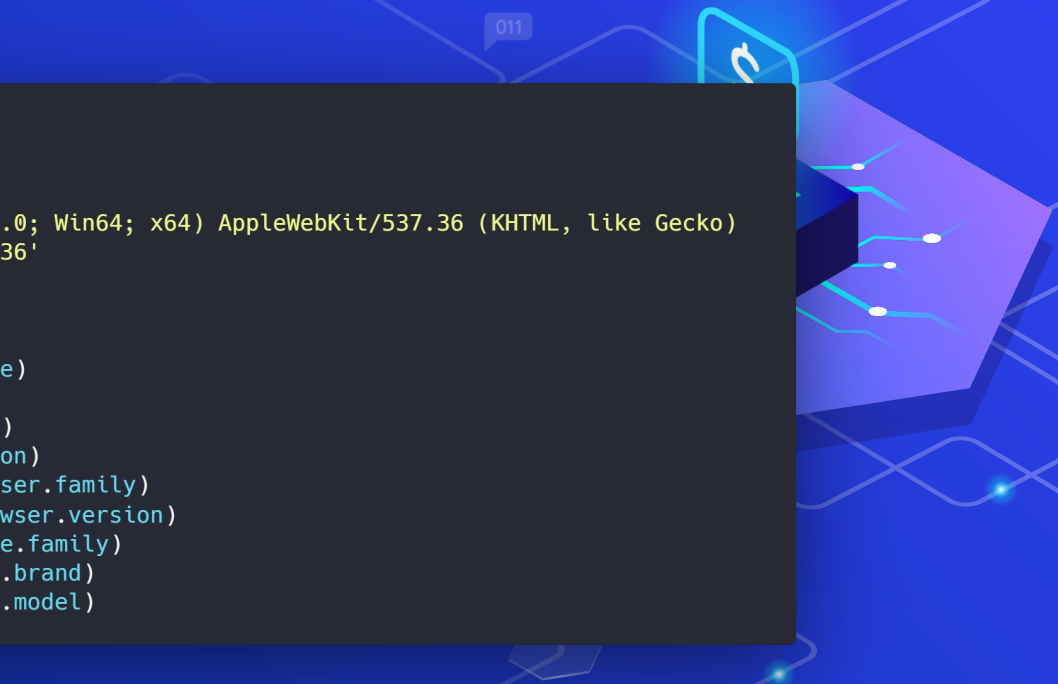
```
import pandas as pd
df = pd.read_csv('Sales_Dec_2019.csv')
df['Purchase_Town'] = df['Purchase Address'].apply(lambda r : r.split(', ')[1]
df['Profit'] = df['Quantity Ordered'] * df['Price Each']
df.groupby('Purchase_Town').sum()['Profit']
```

Agenda

- What is Data
- Machine Learning
- Data Preprocessing
- **Feature Engineering and Extraction**
  - Domain knowledge features
  - Date and Time features
  - String operations
  - **Web Data**
  - Geospatial features
- Feature Transformations
  - Data Cleaning or Cleansing
  - Work with Missing data
  - Work with Categorical data
  - Detect and Handle Outliers
  - Split data to Train and Test Sets
  - Deal with Imbalanced classes
  - Feature Scaling

Web Data

There are a lot of web applications data logs like user agents that contain a lot of info about used browser, OS and device.



```python
import user_agents

ua = 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/83.0.4103.116 Safari/537.36'
ua = user_agents.parse(ua)

print('Is a bot? ', ua.is_bot)
print('Is mobile? ', ua.is_mobile)
print('Is PC? ',ua.is_pc)
print('OS Family: ',ua.os.family)
print('OS Version: ',ua.os.version)
print('Browser Family: ',ua.browser.family)
print('Browser Version: ',ua.browser.version)
print('Device Family: ',ua.device.family)
print('Device Brand: ',ua.device.brand)
print('Device Model: ',ua.device.model)
```

https://deviceatlas.com/blog/list-of-user-agent-strings

Web Data

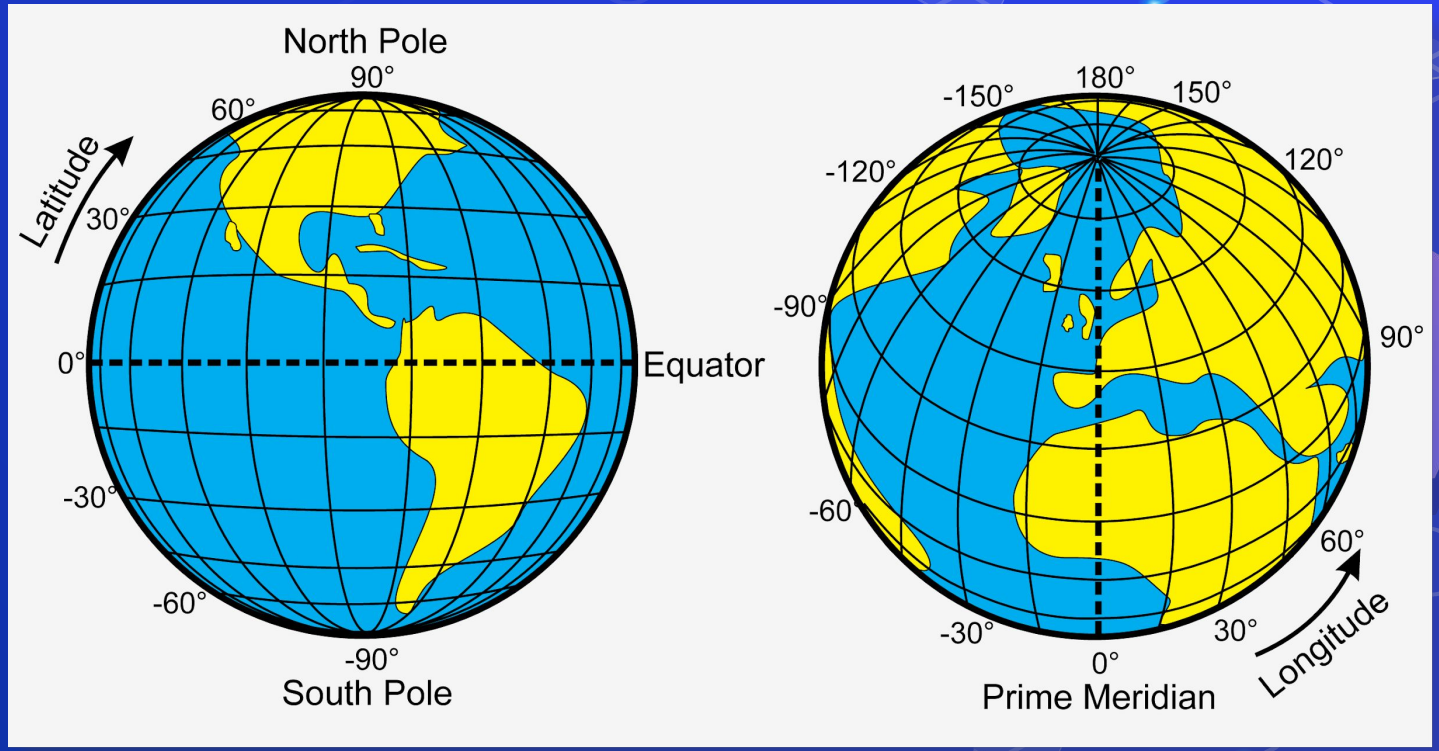
IP address is a great way to extract geographical data from it.

```python
from ip2geotools.databases.noncommercial import DbIpCity as ip2geo


response = ip2geo.get('45.243.72.231', api_key='free')

print(response.ip_address)
print(response.city)
print(response.region)
print(response.country)
print(response.latitude)
print(response.longitude)
```

Agenda

- What is Data
- Machine Learning
- Data Preprocessing
- **Feature Engineering and Extraction**
  - Domain knowledge features
  - Date and Time features
  - String operations
  - Web Data
  - **Geospatial features**
- Feature Transformations
  - Data Cleaning or Cleansing
  - Work with Missing data
  - Work with Categorical data
  - Detect and Handle Outliers
  - Split data to Train and Test Sets
  - Deal with Imbalanced classes
  - Feature Scaling

Geospatial features

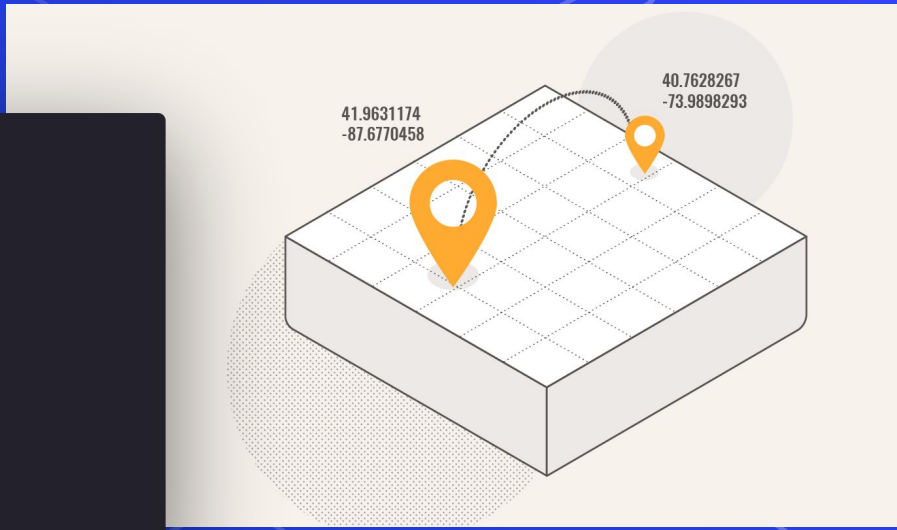


Geospatial features

Measure distance with `great_circle_distance`

https://en.wikipedia.org/wiki/Great-circle_distance

```python
from geopy.distance import great_circle

my_home = (30.109919, 31.308797) # (lat, long)
my_cafe = (30.120982, 31.322026)

great_circle(my_home, my_cafe).kilometers

"""
1.7698508016026915
"""
```



Geospatial features

You can also use Geocoding features to geolocate an address to coordinates, or you Can get the reverse address city and country from coordinates.

https://pypi.org/project/geopy

https://github.com/thampiman/reverse-geocoder

```python
from geopy.geocoders import Nominatim

geolocator = Nominatim(user_agent="Data Science Course")

location = geolocator.geocode("Cairo Festival, Cairo, Egypt")
print(location.raw)

location = geolocator.reverse("29.9812, 31.4289")
print(location.raw)
```

Task 1

Read the “melb_data.csv”
1- take a sample of 200 row.
2- Find the sub city from access the suburb key in the address key in the returned dictionary

Task 1 Solution

```
import pandas as pd
from geopy.geocoders import Nominatim
gelocator = Nominatim(user_agent='Epsilon')
df = pd.read_csv('dastasets/melb_data.csv')
df = df.sample(200)
def get_sub_city(x):
    try:
        Lattitude = x['Lattitude']
        Longitude = x['Longtitude']
        location = gelocator.reverse(f"{Lattitude},{Longitude}")
        return location.raw['address']['suburb']
    except:
        return 'other'
df['Country'] = df.apply(get_sub_city,axis=1)
```

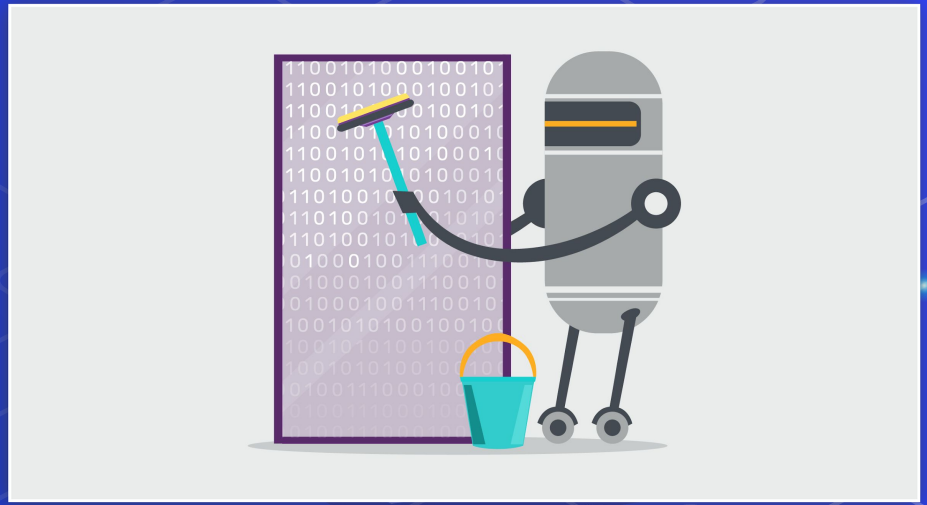
Agenda

- What is Data
- Machine Learning
- Data Preprocessing
- Feature Engineering and Extraction
  - Domain knowledge features
  - Date and Time features
  - String operations
  - Web Data
  - Geospatial features
- **Feature Transformations**
  - **Data Cleaning or Cleansing**
  - Work with Missing data
  - Work with Categorical data
  - Detect and Handle Outliers
  - Split data to Train and Test Sets
  - Deal with Imbalanced classes
  - Feature Scaling

Data Cleaning or Cleansing

Cleaning data is the process of preparing the dataset for analysis. It is very important because the accuracy of machine learning or data mining models are affected because of poor quality of data.

So, data scientists spend a large amount of their time cleaning the dataset and transform them into a format with which they can work with. In fact, data scientists spend 80% of their time cleaning the data.



Example of problems

- Columns can have missing data indicators like xx and ?.
- Height column can have value of 0.
- Weight column can have negative values.
- …

Task 1

Read the dataset “black_Friday.csv”

1- Modify the column of Product_id by removing the “P00”

2- Modify the column of Gender by replacing the questions marks and other wrong values with NaN values.

Task 1 Solution

```
import pandas as pd
import numpy as np
df = pd.read_csv('black_friday.csv')
def mod_prod_id(x):
    return x[3:]
df['product'] = df['Product_ID'].apply(mod_prod_id)
def mod_gender(x):
    if x in['?','xxx']:
        return np.nan
    else:
        return x
df['Gender'] = df['Gender'].apply(mod_gender)
```

Agenda

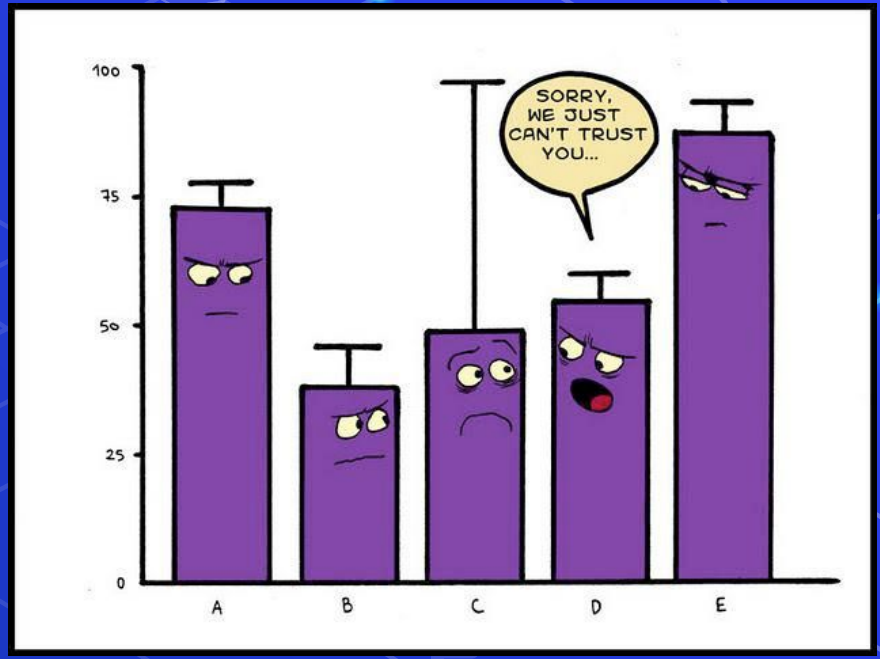
- What is Data
- Machine Learning
- Data Preprocessing
- Feature Engineering and Extraction
  - Domain knowledge features
  - Date and Time features
  - String operations
  - Web Data
  - Geospatial features
- **Feature Transformations**
  - Data Cleaning or Cleansing
  - **Detect and Handle Outliers**
  - Work with Missing data
  - Work with Categorical data
  - Split data to Train and Test Sets
  - Deal with Imbalanced classes
  - Feature Scaling

Detect and Handle Outliers

In statistics, an outlier is a data point that differs significantly from other observations.

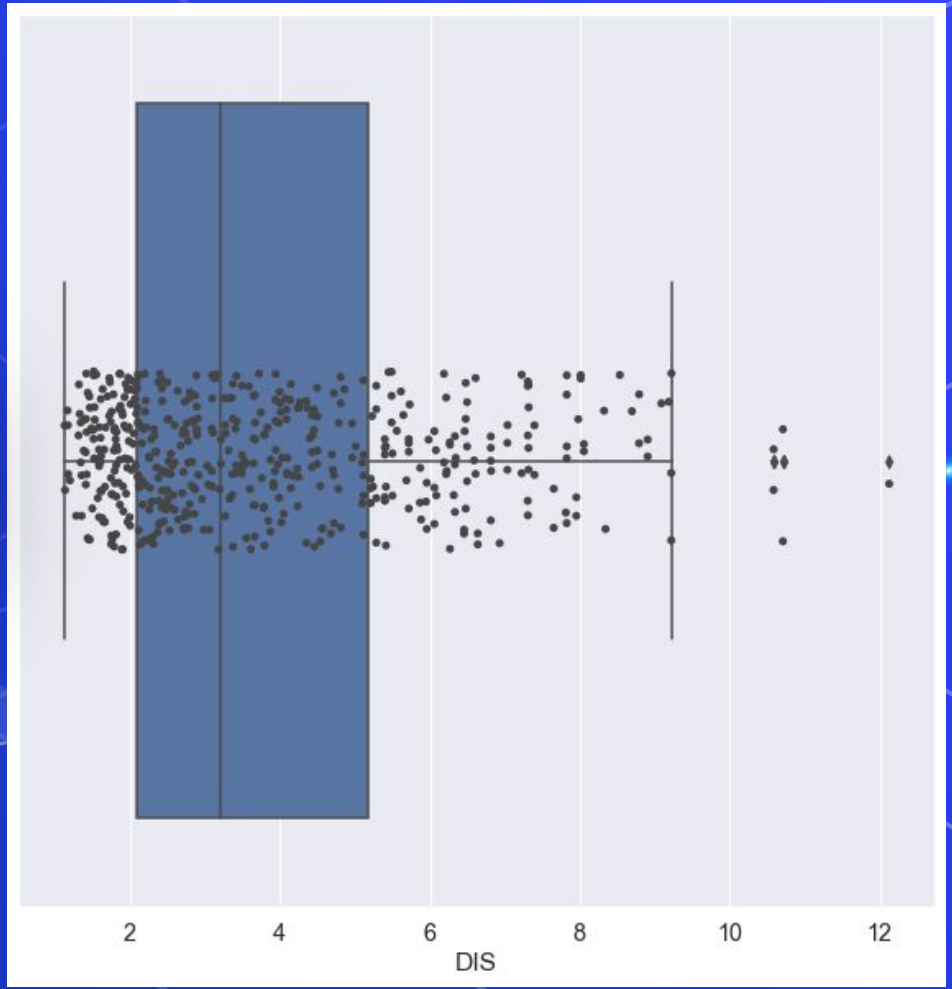
An outlier may be due to variability in the measurement or it may indicate experimental error, the latter are sometimes excluded from the data set.

An outlier can cause serious problems in statistical analyses.



Detect Outliers

```
1 sns.boxplot(x='DIS', data=df)
2 sns.stripplot(x='DIS', data=df)
```



- **Detect Outliers with Visualization.**
- Detect and Handle Outliers with IQR.
- Handle Outliers with business value

Detect and Handle Outliers

```python
from datasist.structdata import detect_outliers

# remove outliers
outliers_indices = detect_outliers(df, 0, df.columns)
df.drop(outliers_indices, inplace=True)


# replace outliers with median value for each column
for col in df.columns:
    outliers_indices = detect_outliers(df, 0, [col])
    col_median = df[col].median()
    df[col].iloc[outliers_indices] = col_median
```

- Detect Outliers with Visualization.
- **Detect and Handle Outliers with IQR.**
- Handle Outliers with business value


011
001
010
100

Detect and Handle Outliers with Business value

- Essentially, instead of removing outliers from the data.
- you change their values to something more representative of your data set , value related to the business of your data , that called business value.
- For example **the "Alice in Wonderland"** movie duration in this dataset is 557 Minutes which is an outlier , so we can replace this outlier value with its real duration 75 Minute

	title	year	certificate	runtime_min	genre
490	Alice in Wonderland	1951	PG-13	557	Animation, Adventure, Comedy
262	The Beatles: Get Back	2021	PG-13	468	Documentary, Biography, Music
167	Guardians of the Galaxy Vol. 2	2017	PG-13	23	Action, Adventure, Comedy
474	Willow	2022–	PG-13	46	Action, Adventure, Drama
14	Eternals	2021	PG-13	156	Action, Adventure, Fantasy
...	...	...	...	...	...
628	All About Steve	2009	PG-13	99	Comedy, Romance
638	Ghost Rider: Spirit of Vengeance	2011	PG-13	96	Action, Fantasy, Thriller
706	Kim Possible	2019 TV Movie	PG-13	86	Action, Adventure, Comedy
1355	In the Mix	2005	PG-13	95	Comedy, Crime, Drama
348	The Wolverine	2013	PG-13	0	Action, Sci-Fi

- Detect Outliers with Visualization.
- Detect and Handle Outliers with IQR.
- **Handle Outliers with business value**

Agenda

- What is Data
- Machine Learning
- Data Preprocessing
- Feature Engineering and Extraction
  - Domain knowledge features
  - Date and Time features
  - String operations
  - Web Data
  - Geospatial features
- **Feature Transformations**
  - Data Cleaning or Cleansing
  - Detect and Handle Outliers
  - **Work with Missing data**
  - Work with Categorical data
  - Split data to Train and Test Sets
  - Deal with Imbalanced classes
  - Feature Scaling

Work with Missing data

```
1 df.isnull().sum()
```

- **Check Missing Data**
- Drop Missing Data
- Fill Missing data with Pandas
- Fill Missing data with Sklearn `SimpleImputer`
- Fill Missing data with Sklearn `KNNImputer`

Check missing data

In Pandas missing data is represented by two value: None/NaN

we use a function isnull() to check even if NAN is true/false

we use a function isnull() to check number of nan in each column

```
[3] import pandas as pd
    #read csv file
    data=pd.read_csv('/content/data-2010-12-01.csv')
    data.head(6)
```

	X	Y	Z	W
0	10	12.0	1.0	5.0
1	27	NaN	5.0	NaN
2	20	39.0	0.0	2.0
3	17	11.0	NaN	NaN
4	93	15.0	6.0	28.0
5	59	0.0	NaN	NaN

```
data.isnull()
```

	X	Y	Z	W
0	False	False	False	False
1	False	True	False	True
2	False	False	False	False
3	False	False	True	True
4	False	False	False	False
5	False	False	True	True

```
data.isnull().sum()
```

```
X    0
Y    1
Z    2
W    3
dtype: int64
```

Drop Missing Data

1.When the data goes missing on 70–80 percent of the variable:

dropping the variable should be considered

2. When the data goes missing on 1–5 percent of the variable

dropping missing values only should be considered

dropna(axis=0, how="any", subset=None, inplace=False)

axis: possible values are {0 or ‘index’, 1 or ‘columns’}, default 0. If 0, drop rows with null values. If 1, drop columns with missing values.

how: possible values are {‘any’, ‘all’}, default ‘any’. If ‘any’, drop the row/column if any of the values is null. If ‘all’, drop the row/column if all the values are missing.

subset: specifies the rows/columns to look for null values

Dropping rows/columns with at least 1 null value

```
[3] import pandas as pd
    #read csv file
    data=pd.read_csv('/content/data-2010-12-01.csv')
    data.head(6)
```

	X	Y	Z	W
0	10	12.0	1.0	5.0
1	27	NaN	5.0	NaN
2	20	39.0	0.0	2.0
3	17	11.0	NaN	NaN
4	93	15.0	6.0	28.0
5	59	0.0	NaN	NaN

```
data.dropna(axis=0)
```

	X	Y	Z	W
0	10	12.0	1.0	5.0
2	20	39.0	10.0	2.0
4	93	15.0	6.0	28.0

Rows

```
data.dropna(axis=1)
```

	X
0	10
1	27
2	20
3	17
4	93
5	59

columns

Work with Missing data

```
1 df.dropna(axis = 0)
2
3 df.dropna(axis = 1)
```

- Check Missing Data
- **Drop Missing Data**
- Fill Missing data with Pandas
- Fill Missing data with Sklearn `SimpleImputer`
- Fill Missing data with Sklearn `KNNImputer`

Fill Missing data with Pandas

1. Filling the missing data with the mean or median value if it’s a numerical variable.
2. Filling the missing data with the median value only if it’s a numerical variable with outliers
3. Filling the missing data with mode if it’s a categorical value.

Work with Missing data

```python
# fill with mean
df['BuildingArea'].fillna(df['BuildingArea'].mean(), inplace=True)


# fill with median
df['YearBuilt'].fillna(df['YearBuilt'].median(), inplace=True)


# fill with mode (most frequent)
df['Car'].fillna(df['Car'].mode()[0], inplace=True)
```

- Check Missing Data
- Drop Missing Data
- **Fill Missing data with Pandas**
- Fill Missing data with Sklearn `SimpleImputer`
- Fill Missing data with Sklearn `KNNImputer`

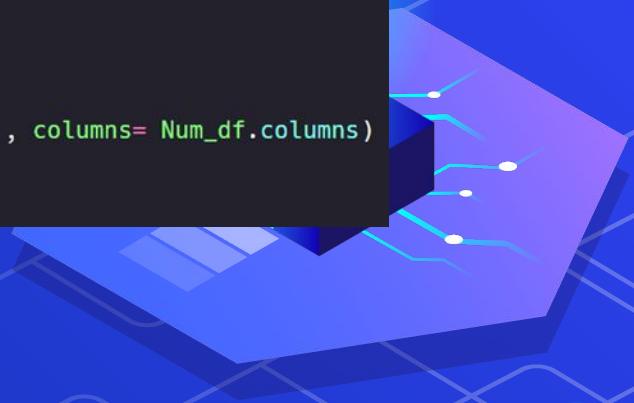
Work with Missing data

```python
from sklearn.impute import SimpleImputer

# define the Imputer properties
imputer = SimpleImputer(strategy='mean')    # can use 'median' or 'most_frequent'

# impute
df['BuildingArea'] = imputer.fit_transform(df[['BuildingArea']])

# get the filled value
imputer.statistics_
```

- Check Missing Data
- Drop Missing Data
- Fill Missing data with Pandas
- **Fill Missing data with Sklearn `SimpleImputer`**
- Fill Missing data with Sklearn `KNNImputer`

Work with Missing data

```
import pandas as pd
from sklearn.impute import KNNImputer
df = pd.read_csv('melb_data.csv'

imputer = KNNImputer()
Num_df = df.select_dtypes(include = "number")
Cat_df = df.select_dtypes(include = "object_")
Num_df = pd.DataFrame(imputer.fit_transform(Num_df) , columns= Num_df.columns)

df = pd.concat([Num_df , Cat_df] , axis = 1)
```



- Check Missing Data
- Drop Missing Data
- Fill Missing data with Pandas
- Fill Missing data with Sklearn `SimpleImputer`
- **Fill Missing data with Sklearn `KNNImputer`**

Task 2

- Read the dataset “friends_mod.csv”

1. **Impute the missing values in “height(cm)” column**
2. **Impute the missing values in “section” column**
3. **Save the dataset after modification with**

df.to_csv(“friends_mod.csv” , index = False)

Please take care of outliers and unclean in the two columns.

Task 2 Solution

```
import pandas as pd
import seaborn as sns

df = pd.read_csv('friends_modified.csv',na_values=['xx','NaN'])

sns.boxplot(data = df , x ='height(cm)')
df['height(cm)'].fillna(df['height(cm)'].median(),inplace=True)

df['section'].fillna(df['section'].mode()[0],inplace=True)
```

Agenda

- What is Data
- Machine Learning
- Data Preprocessing
- Feature Engineering and Extraction
  - Domain knowledge features
  - Date and Time features
  - String operations
  - Web Data
  - Geospatial features
- **Feature Transformations**
  - Data Cleaning or Cleansing
  - Detect and Handle Outliers
  - Work with Missing data
  - **Work with Categorical data**
  - Split data to Train and Test Sets
  - Deal with Imbalanced classes
  - Feature Scaling

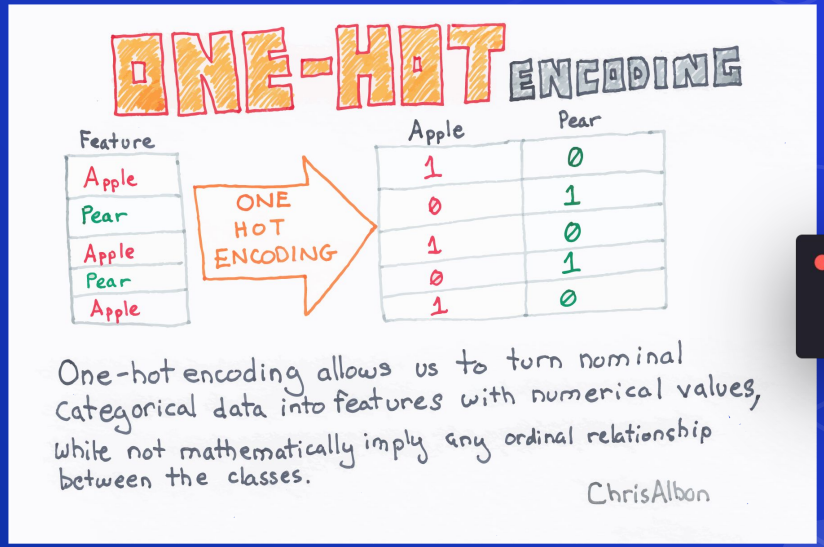
Work with Categorical ordinal data (Label Encoder)

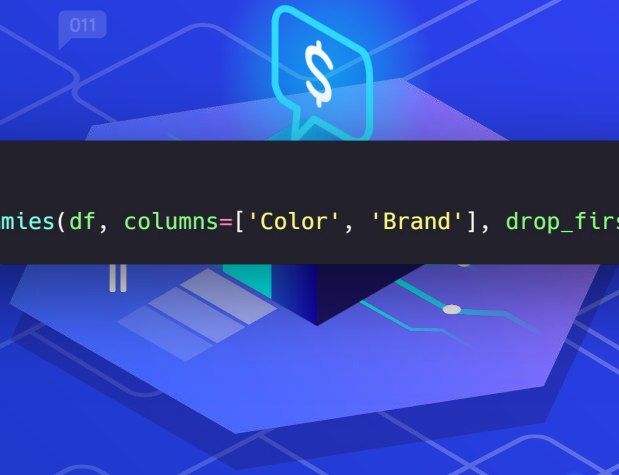
- Label encoding converts the data in machine-readable form, but it assigns a unique number(starting from 0) to each class of data , in a sequence ordinal manner.
- To use label encoding, your data should must have inherent order in its logic
- We can’t use it with nominal data (non – ordered).
- An attribute having output classes Mexico, Paris, Dubai. On Label Encoding, this column lets Mexico is replaced with 0, Paris is replaced with 1, and Dubai is replaced with 2.
- With this, it can be interpreted that Dubai has high priority than Mexico and Paris while training the model, But actually, there is no such priority relation between these cities here.

```python
size_dict = {'XS':1,
             'S':2,
             'M':3,
             'L':4,
             'XL':5,
             'XXL':6}

# apply using map
df['Size'] = df['Size'].map(size_dict)
```

- **Work with Ordinal Features with pandas map method.**

Work with Categorical Nominal data (ONE-HOT Encoder / Binary Encoding)

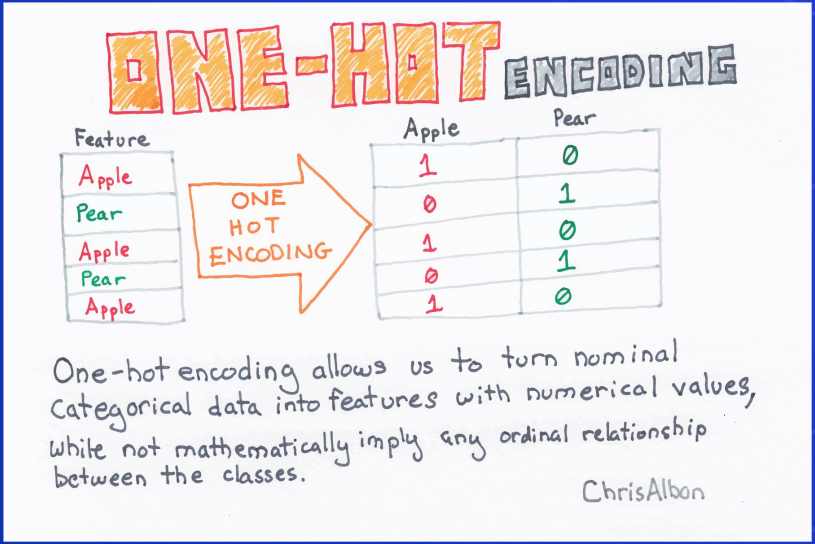




```
df = pd.get_dummies(df, columns=['Color', 'Brand'], drop_first=True)
```

- Work with Ordinal Features with pandas `map` method.
- **Work with Nominal Features with pandas `get_dummies` method.**
- Work with Nominal Features with sklearn `OneHotEncoder` method.

Work with Categorical Nominal data (ONE-HOT Encoder / Binary Encoding)




```python
import pandas as pd
from sklearn.preprocessing import OneHotEncoder
df = pd.read_csv('t-shirts.csv')

Encoder = OneHotEncoder(sparse=False)
Transformed_Color = Encoder.fit_transform(df[['Color']])
Transformed_Color_Df = pd.DataFrame(Transformed_Color , columns= Encoder.get_feature_names())

df = pd.concat([df,Transformed_Color_Df] , axis = 1 )
df.drop('Color' , axis = 1 , inplace =True)
```

- Work with Ordinal Features with pandas `map` method.
- Work with Nominal Features with pandas `get_dummies` method.
- **Work with Nominal Features with sklearn `OneHotEncoder` method.**

Work with Categorical data

col1	col2
First	animal
third	plant
second	country

- What is the kind of preprocessing ?

Short Answer

Task 3

- Read the dataset “friends_mod.csv” that you save it in task 1

1. **Make a new column called “gender” from the column “age_sex” with the appropriate feature engineering.**
2. **Make the appropriate type of encoding to the categorical columns (“section”,”gender”)**

Task 3 Solution

```
import pandas as pd
import seaborn as sns

df = pd.read_csv('friends_modified.csv',na_values=['xx','NaN'])

df['gender'] = df['age_sex'].apply(lambda r : r.split('_')[-1])

dec_sec = {
        'A' : 1,
        'B' : 2,
        'C' : 3
          }
df['section'] = df['section'].map(dec_sec)

df['gender'] = pd.get_dummies(df['gender'],drop_first=True)
```

Binary Encoding for Nominal Data

- The categories are encoded in Ordinal Numeric form.
- Then those integers are converted into binary code, so for example 5 becomes 101 and 10 becomes 1010
- Then the digits from that binary string are split into separate columns.
- Each observation is encoded across the columns in its binary form.
- Example: if we have Country Feature with 16 different countries:
  - By One-Hot Encoding it will result in 16 columns.
  - By Binary Encoding, it will result only in 4 columns

```
X = pd.DataFrame(list('abcdefghijklmnop'))
X
```

	0
0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

```
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder(sparse=False)
countries= ohe.fit_transform(X.values.reshape(-1,1)).astype(int)
pd.DataFrame(countries)
```

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
3	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
5	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
6	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
9	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
10	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
11	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
12	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
13	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
14	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

```
import category_encoders as ce

ce_bin = ce.BinaryEncoder()
ce_bin.fit_transform(X)
```

	0_0	0_1	0_2	0_3	0_4
0	0	0	0	0	1
1	0	0	0	1	0
2	0	0	0	1	1
3	0	0	1	0	0
4	0	0	1	0	1
5	0	0	1	1	0
6	0	0	1	1	1
7	0	1	0	0	0
8	0	1	0	0	1
9	0	1	0	1	0
10	0	1	0	1	1
11	0	1	1	0	0
12	0	1	1	0	1
13	0	1	1	1	0
14	0	1	1	1	1
15	1	0	0	0	0

Agenda

- What is Data
- Machine Learning
- Data Preprocessing
- Feature Engineering and Extraction
  - Domain knowledge features
  - Date and Time features
  - String operations
  - Web Data
  - Geospatial features
- **Feature Transformations**
  - Data Cleaning or Cleansing
  - Detect and Handle Outliers
  - Work with Missing data
  - Work with Categorical data
  - **Split data to Train and Test Sets**
  - Deal with Imbalanced classes
  - Feature Scaling

Split data to Train and Test Sets

When you're working on a model and want to train it, you obviously have a dataset. But after training, we have to test the model on some test dataset.

the obvious solution is to split the dataset you have into two sets, one for training and the other for testing; and you do this before you start training your model.


```python
from sklearn.model_selection import train_test_split

x = df.drop('healthy', axis=1)
y = df['healthy']

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size = 0.2)
```

Sampling techniques in Train and Test Sets

Important in **imbalanced data.**

Parameter of train_test_split:

1. Shuffle

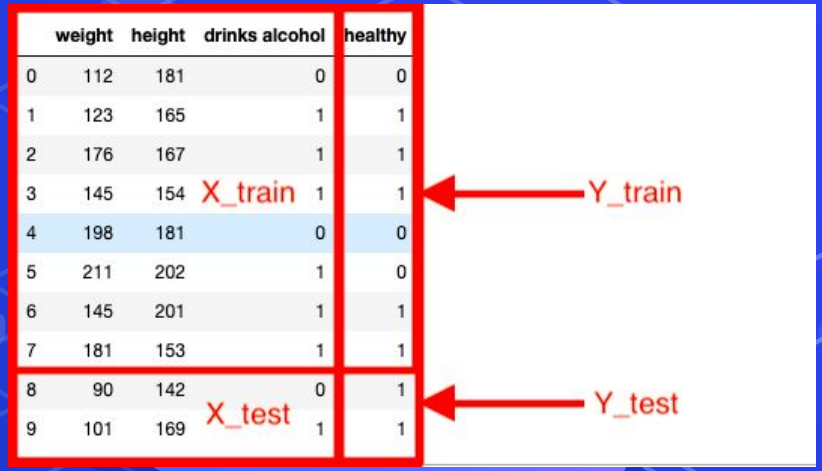
With shuffle=True, data is shuffled then the split happens.

2. Stratify

Makes a split so that a proportion of the values in the sample produced will be the same as the proportion of values provided to parameter stratify.

3. Random state

If added you ensure that the split remains the same whenever you run the code



	weight	height	drinks alcohol	healthy
0	112	181	0	0
1	123	165	1	1
2	176	167	1	1
3	145	154	1	1
4	198	181	0	0
5	211	202	1	0
6	145	201	1	1
7	181	153	1	1
8	90	142	0	1
9	101	169	1	1

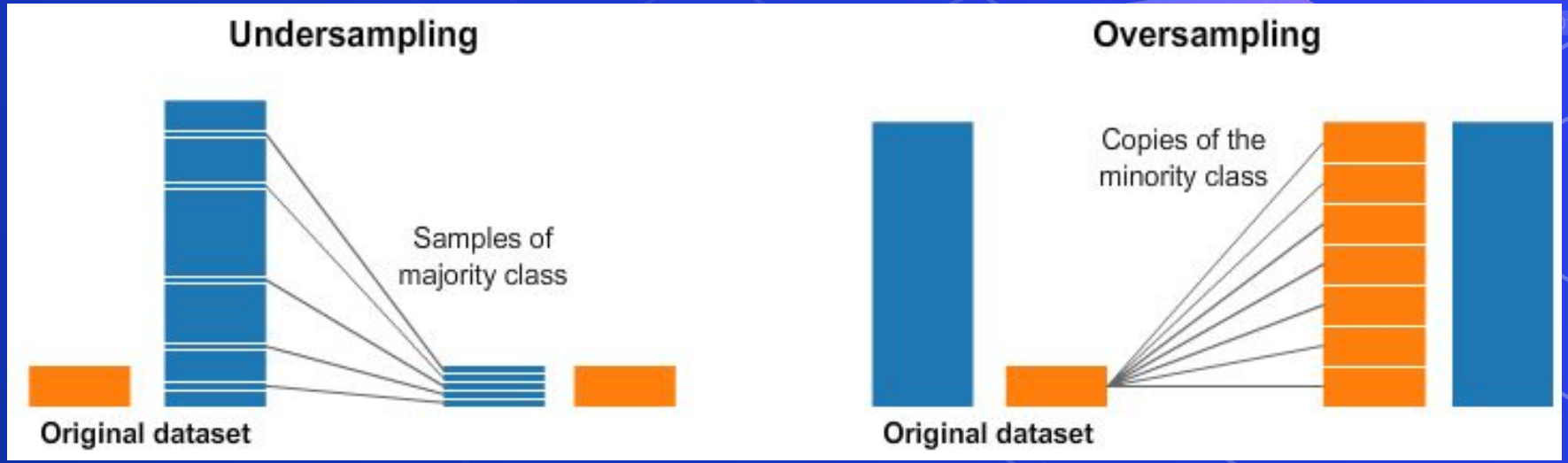
```
from sklearn.model_selection import train_test_split

x = df.drop('healthy', axis=1)
y = df['healthy']

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size = 0.2
                                                    , shuffle =True, stratify = y)
```

Agenda

- What is Data
- Machine Learning
- Data Preprocessing
- Feature Engineering and Extraction
  - Domain knowledge features
  - Date and Time features
  - String operations
  - Web Data
  - Geospatial features
- **Feature Transformations**
  - Data Cleaning or Cleansing
  - Work with Missing data
  - Work with Categorical data
  - Detect and Handle Outliers
  - Split data to Train and Test Sets
  - **Deal with Imbalanced classes**
  - Feature Scaling

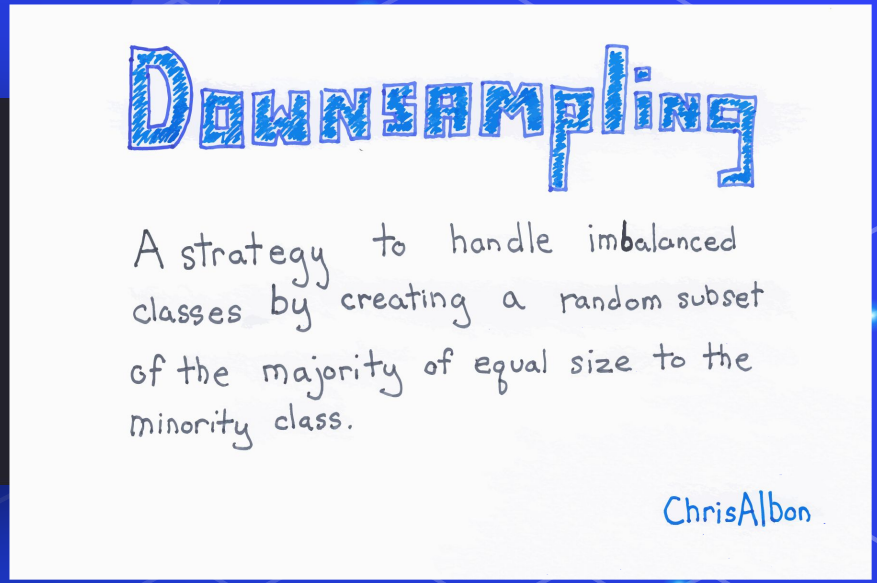
Deal with Imbalanced classes

Imbalanced classes are a common problem in machine learning classification where there are an unbalanced ratio of observations in each class. Class imbalance can be found in many different areas including medical diagnosis, spam filtering, and fraud detection.



Deal with Imbalanced classes

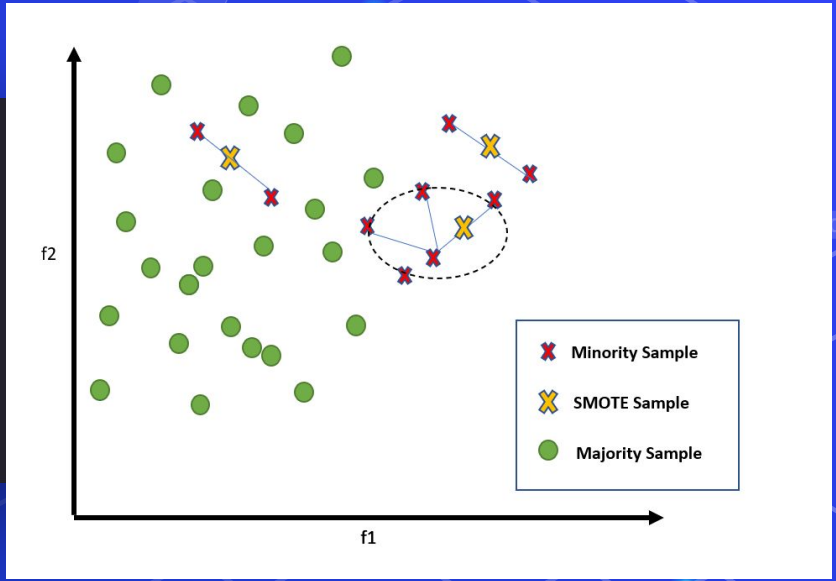
```python
from imblearn.under_sampling import RandomUnderSampler

x = df.drop('Class', axis=1)
y = df['Class']

rus = RandomUnderSampler()

x_train, y_train = rus.fit_sample(x_train, y_train)
```



- **Down-sampling or Under-sampling Majority Class**
- Generate Synthetic Samples with SMOTE

Deal with Imbalanced classes

```python
from imblearn.over_sampling import SMOTE

x = df.drop('Class', axis=1)
y = df['Class']

smote = SMOTE()

x_train, y_train = smote.fit_sample(x_train, y_train)
```



- Down-sampling or Under-sampling Majority Class
- **Generate Synthetic Samples with SMOTE**

Deal with Imbalanced classes (Combination)

- Oversampling methods duplicate or create new synthetic examples in the minority class, whereas under sampling methods delete or merge examples in the majority class.
- Both types of resampling can be effective when used in isolation, although can be more effective when both types of methods are used together.

Deal with Imbalanced classes

```
# Manually Combine Over and Undersampling Methods
from imblearn.over_sampling import RandomOverSampler from
imblearn.under_sampling import RandomUnderSampler from imblearn.pipeline import
Pipeline
#over = RandomOverSampler(sampling_strategy=0.1) over =
SMOTE(sampling_strategy=0.1) under = RandomUnderSampler(sampling_strategy=0.5)
pipeline = Pipeline(steps= [('o', over), ('u', under)]) # fit and apply the
pipeline X_resampled, y_resampled = pipeline.fit_resample(X, y)
```

What types of preprocessing will be used here??

A	B	class
2400	Egypt	1
2500	France	1
2800	England	1
	USA	0
32750	Libya	1

Agenda

- What is Data
- Machine Learning
- Data Preprocessing
- Feature Engineering and Extraction
  - Domain knowledge features
  - Date and Time features
  - String operations
  - Web Data
  - Geospatial features
- **Feature Transformations**
  - Data Cleaning or Cleansing
  - Work with Missing data
  - Work with Categorical data
  - Detect and Handle Outliers
  - Split data to Train and Test Sets
  - Deal with Imbalanced classes
  - **Feature Scaling**

Feature Scaling

- It is common to scale data prior to fitting a machine learning model.
- This is because data often consists of many different input variables or features (columns) and each may have a different range of values or units of measure, such as feet, miles, kilograms, dollars, etc.
- If there are input variables that have very large values relative to the other input variables, these large values can dominate or skew some machine learning algorithms.
- The result is that the algorithms pay most of their attention to the large values and ignore the variables with smaller values.
- This includes algorithms that use a weighted sum of inputs :

1. like linear regression
2. logistic regression
3. artificial neural networks

as well as algorithms that use distance measures between examples such as :

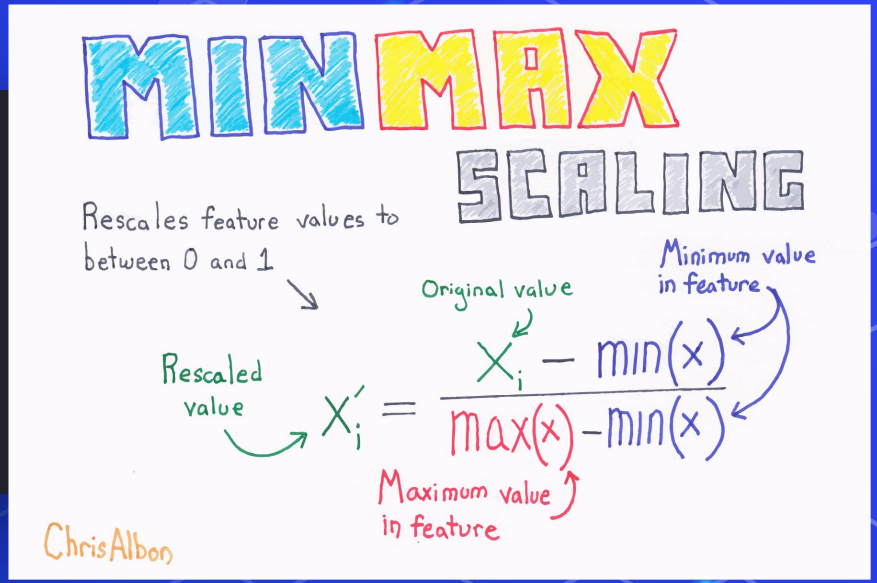
1. k-nearest neighbors
2. support vector machines.

Data Leakage

- Data leakage happens when knowledge of the hold-out test set leaks into the dataset used to train the model.
- This can result in an incorrect estimate of model performance when making predictions on new data.
- A careful application of data preparation techniques is required in order to avoid data leakage.
- For example , in feature scaling you must fit only the train data then transform the train and test data.
- You **mustn't** fit the test data again to avoid data leakage.
- Data leakage can cause serious problems to our machine learning model as it generally gives exceptional accuracy or much better results than it would otherwise in a real-world scenario.
- But after deployment in production, it crashes or performs rather poorly.

Feature Scaling

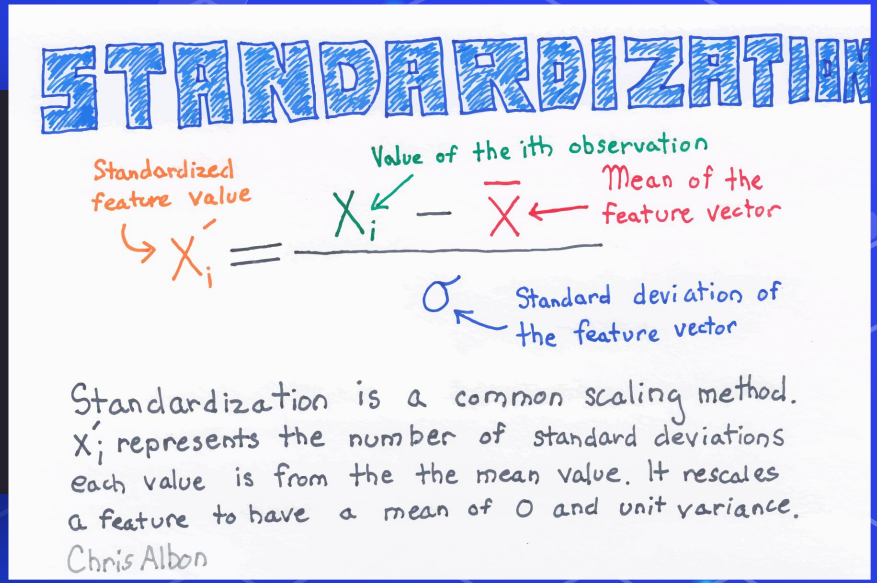
```python
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

scaler.fit(x_train)

scaled_x_train = scaler.transform(x_train)
scaled_x_test = scaler.transform(x_test)
```



- **Normalization with Sklearn `MinMaxScaler`.**
- Standardizing data with `StandardScaler`.

Feature Scaling

```python
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

scaler.fit(x_train)

scaled_x_train = scaler.transform(x_train)
scaled_x_test = scaler.transform(x_test)
```



- Normalization with Sklearn `MinMaxScaler`.
- **Standardizing data with `StandardScaler`.**

Normalization with Sklearn MinMaxScaler.

- it is the simplest method and consists of rescaling the range of features to scale the range in [0, 1]

Standardizing data with StandardScaler.

- Feature standardization makes the values of each feature in the data have zero mean and unit variance.

Feature Scaling (Robust Scalar)

- Sometimes an input variable may have outlier values. These are values on the edge of the distribution that may have a low probability of occurrence yet are overrepresented for some reason.
- Outliers can skew a probability distribution and make data scaling using standardization difficult as the calculated mean and standard deviation will be skewed by the presence of the outliers.
- One approach to standardizing input variables in the presence of outliers is to ignore the outliers from the calculation of the mean and standard deviation, then use the calculated values to scale the variable. This is called robust standardization or **robust data scaling.**
- This can be achieved by calculating the median (50th percentile) and the 25th and 75th percentiles.
- The values of each variable then have their median subtracted and are divided by the interquartile range (IQR) which is the difference between the 75th and 25th percentiles

$$\text{value} = \frac{\text{value} - \text{median}}{p_{75} - p_{25}}$$

Feature Scaling

```
from sklearn.preprocessing import RobustScaler
scaler = RobustScaler()
scaler.fit(train_data)
train_data = scaler.transform(train_data)
test_data= scaler.transform(test_data)
```

- **Robust_Scaler**

What types of preprocessing will be used here??

salary	age	B	class
2400	34	second	0
2500	55	fourth	1
2800	23	ninth	0
7000	27	first	0
32750	43	third	0

Template form

```
# Python Project Template

# 1. Prepare Problem
# a) Load libraries
# b) Load dataset

# 2. Summarize Data
# a) Descriptive statistics
# b) Data visualizations

# 3. Prepare Data
# a) Data Cleaning
# b) Feature Selection
# c) Data Transforms

# 4. Evaluate Algorithms
# a) Split-out validation dataset
# b) Test options and evaluation metric
# c) Spot Check Algorithms
# d) Compare Algorithms

# 5. Improve Accuracy
# a) Algorithm Tuning
# b) Ensembles

# 6. Finalize Model
# a) Predictions on validation dataset
# b) Create standalone model on entire training dataset
# c) Save model for later use
```

1.Prepare Problem

a) Load libraries that you intend to use.

```
# Load libraries
from pandas import read_csv
from pandas.tools.plotting import scatter_matrix
from matplotlib import pyplot
from sklearn.model_selection import train_test_split
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score
```

b) Load dataset from CSV

1. names: featu

2.file name: name.csv

```
# Load dataset
filename = 'iris.data.csv'
names = ['sepal-length', 'sepal-width', 'petal-length', 'petal-width', 'class']
dataset = read_csv(filename, names=names)
```

2. Summarize Data

a) Descriptive statistics

1. **Dimensions of the dataset.**

```
# shape
print(dataset.shape)
```

```
(150, 5)
```

150 rows and 5 columns

2. **Peek at the Data**

```
# head
print(dataset.head(20))
```

see first 20 rows of your data

2. Summarize Data

a) Descriptive statistics

3. Statistical Summary.

This includes the count, mean, the min and max values as well as some percentiles.

```
# descriptions
print(dataset.describe())
```

	sepal-length	sepal-width	petal-length	petal-width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

2. Summarize Data

a) Descriptive statistics

4. Class Distribution.

to check if class is balanced distribution or not

```
# class distribution
print(dataset.groupby('class').size())
```

2. Summarize Data

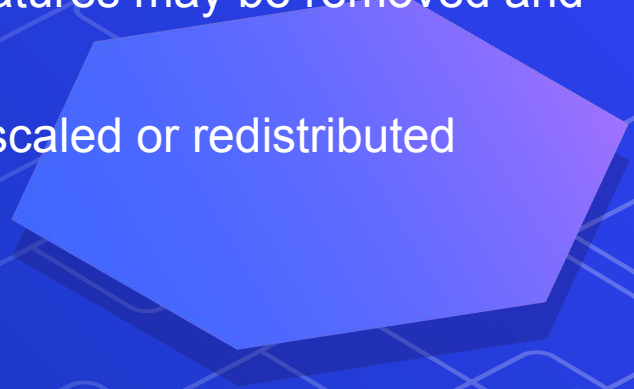
b) Data Visualization

1. Univariate plots to better understand each attribute.
   - plots of each individual variable.by boxplot and histogram

3. Multivariate Plots
   - interactions between the variables by scatterplot

3. prepare Data

- Cleaning data by removing duplicates, marking missing values and even imputing missing values.
- Feature selection where redundant features may be removed and new features developed
- Data transforms where attributes are scaled or redistributed



4. Evaluate Algorithms

1. Separate out a validation dataset.

2. Setup the test harness to use 10-fold cross validation.

3. Build 5 different models to predict species from flower measurements(KNN, CART, NB,LR)

4. Select the best model.

5. Improving accuracy

1) Hyperparameters tuning

1) Ensemble by bagging ,boosting or voting ensemble

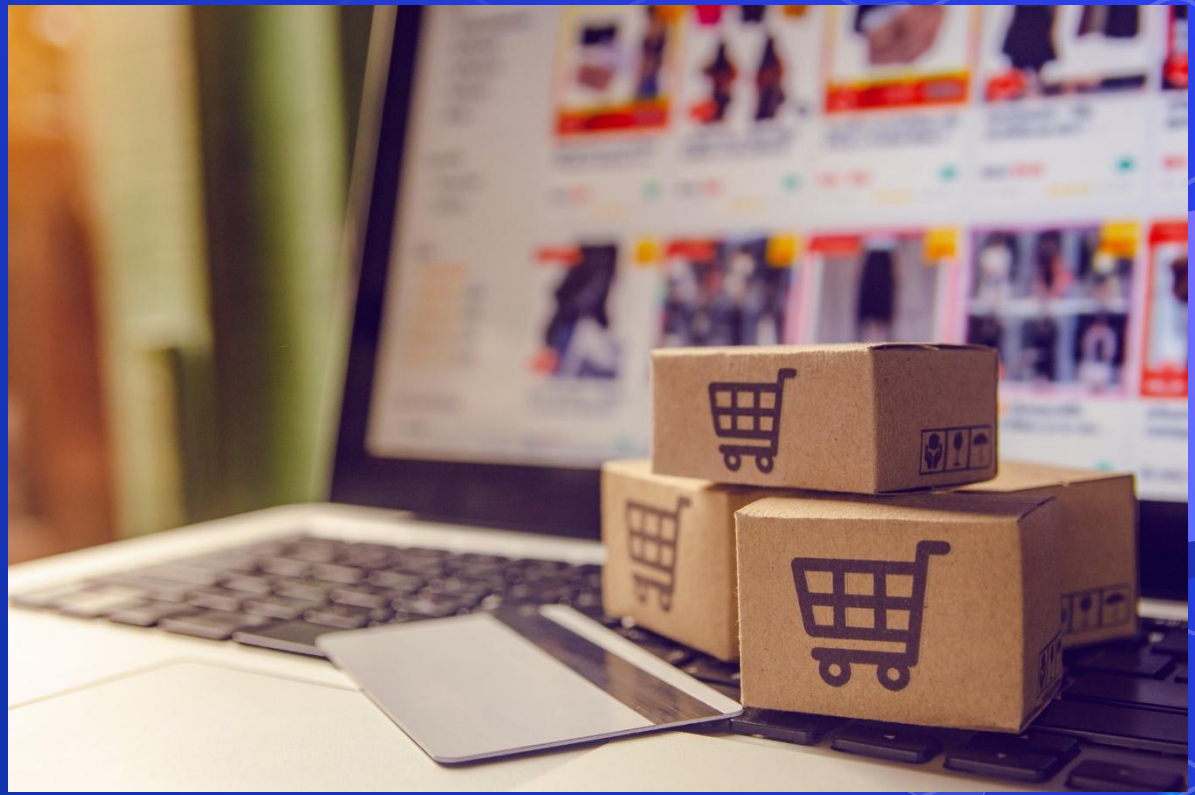
Project #17 Data Analyst Jobs Analysis

Project #18 Google Play Store

o Project #19 Uber Analysis

Project #20 (Sales product data Analysis)

Project #21 (Ecommerce System data Analysis)



Project #22 (Netflix data Analysis)

Questions ?!

Thanks!

>_ Live long and prosper